

Name:Sudiptika J Register No:250701754

Experiment Name:Handling Missing and Inappropriate Data in a Dataset

```
import numpy as np
import pandas as pd
df = pd.read_csv("Hotel_Dataset.csv")
print("--- Initial Duplicate Identification ---")
print(df.duplicated())
print("\n--- Dataframe Info Summary ---")
df.info()
df.drop_duplicates(inplace=True) index =
np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
df.drop(["Age_Group.1"], axis=1, inplace=True)
df.CustomerID.loc[df.CustomerID < 0] = np.nan
df.Bill.loc[df.Bill < 0] = np.nan
df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
df["NoOfPax"].loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20)] = np.nan
print("\n--- Unique Categorical Value Sets ---") print("Age Groups:",
df.Age_Group.unique()) print("Hotels:", df.Hotel.unique())
df.Hotel.replace(["Ibys"], "Ibis", inplace=True)
df.FoodPreference.replace(["Vegetarian", "veg"], "Veg", inplace=True)
df.FoodPreference.replace(["non-Veg"], "Non-Veg", inplace=True)
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),
inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
df["Rating(1-5)"].fillna(round(df["Rating(1-5)"].median()),
inplace=True)
df.Bill.fillna(round(df.Bill.mean()), inplace=True)
print("\n--- Final Cleaned Dataset (df) ---")
print(df)
--- Initial Duplicate Identification ---
0      False
1      False
2      False
3      False
4      False
5      False
6      False
7      False
8      False
9       True
10     False
dtype: bool
--- Dataframe Info Summary ---
<class 'pandas.core.frame.DataFrame'>
```


RangeIndex: 11 entries, 0 to 10

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	CustomerID	11 non-null	int64
1	Age_Group	11 non-null	object
2	Rating(1-5)	11 non-null	int64
3	Hotel	11 non-null	object
4	FoodPreference	11 non-null	object
5	Bill	11 non-null	int64
6	NoOfPax	11 non-null	int64
7	EstimatedSalary	11 non-null	int64
8	Age_Group.1	11 non-null	object

dtypes: int64(5), object(4)
memory usage: 924.0+ bytes

--- Unique Categorical Value Sets ---

Age Groups: ['20-25' '30-35' '25-30' '35+']

Hotels: ['Ibis' 'LemonTree' 'RedFox' 'Ibys']

--- Final Cleaned Dataset (df) ---

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill
0	1.0	20-25	4	Ibis	Veg	1300.0
1	2.0	30-35	5	LemonTree	Non-Veg	2000.0
2	3.0	25-30	6	RedFox	Veg	1322.0
3	4.0	20-25	-1	LemonTree	Veg	1234.0
4	5.0	35+	3	Ibis	Veg	989.0
5	6.0	35+	3	Ibis	Non-Veg	1909.0
6	7.0	35+	4	RedFox	Veg	1000.0
7	8.0	20-25	7	LemonTree	Veg	2999.0
8	9.0	25-30	2	Ibis	Non-Veg	3456.0
9	10.0	30-35	5	RedFox	Non-Veg	1801.0

	NoOfPax	EstimatedSalary
0	2.0	40000.0
1	3.0	59000.0
2	2.0	30000.0
3	2.0	120000.0
4	2.0	45000.0

5	2.0	122220.0
6	2.0	21122.0
7	2.0	345673.0
8	3.0	96755.0
9	4.0	87777.0

C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:12:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0! You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df.CustomerID.loc[df.CustomerID < 0] = np.nan
```

C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:12:
SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df.CustomerID.loc[df.CustomerID < 0] = np.nan
```

C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:13:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0! You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the

assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df.Bill.loc[df.Bill < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:13:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df.Bill.loc[df.Bill < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:14:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0! You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy. A typical
example is when you are setting values in a column of a
DataFrame, like:
```

```
df["col"][row_indexer] = value
```

Use `df.loc[row\_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:14:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:15:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0! You are setting values through chained assignment. Currently this
```

works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["NoOfPax"].loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20)] =  
np.nan
```

C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:15:
SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["NoOfPax"].loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20)] =  
np.nan
```

C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:19:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `df[col].method(value, inplace=True)`, try using `df.method({col: value}, inplace=True)` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
df.Hotel.replace(["Ibys"], "Ibis", inplace=True)
```

C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:20:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df.FoodPreference.replace(["Vegetarian", "veg"], "Veg",
inplace=True)
```

C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:24:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df["Rating(1-5)"].fillna(round(df["Rating(1-5)"].median()),
inplace=True)
```